```
using Newtonsoft.Json;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Globalization;
using System.Linq;

namespace SistemaGestionCGI.Settings
{
    public class ConnectionSqlServer
    {
        public string Server { get; set; }
        public string Database { get; set; }
        public static ConnectionSqlServer Instance { get; } = new
ConnectionSqlServer();
        private readonly NumberFormatInfo nfi = new NumberFormatInfo {
NumberDecimalSeparator = "." };

        private string CreateConnectionString()
        {
            // Se mantiene la cadena base, permitiendo formateo dinámico si
Server/Database se asignan
            string baseConn = @"server=DESKTOP-
A925LIU\SQLEXPRESS2019;database=INVESTIGACION;INTEGRATED SECURITY=true";
            return (string.IsNullOrEmpty(Server) || string.IsNullOrEmpty(Database))
                ? baseConn
                : string.Format("server={0};database={1};INTEGRATED SECURITY=true",
Server, Database);
        }

        // --- MÉTODOS DE APOYO CENTRALIZADOS ---

        private T ExecuteBase<T>(string sql, Func<SqlCommand, T> action)
        {
            using (SqlConnection conn = new SqlConnection(CreateConnectionString()))
            {
                try
                {
                    conn.Open();
                    using (SqlCommand cmd = new SqlCommand(sql, conn))
                    {
                        cmd.CommandTimeout = 60;
                        return action(cmd);
                    }
                }
                catch (SqlException ex) { throw new Exception("Error SQL: " +
ex.Message); }
                catch (Exception ex) { throw new Exception("Error General: " +
ex.Message); }
            }
        }

        private ArrayList ReadToArrayList(SqlCommand cmd)
        {
            ArrayList data = new ArrayList();
            using (SqlDataReader reader = cmd.ExecuteReader())
```

```
            {
                if (reader.HasRows)
                {
                    while (reader.Read())
                    {
                        Hashtable fields = new Hashtable();
                        for (int i = 0; i < reader.FieldCount; i++)
                        {
                            fields.Add(reader.GetName(i),
GetTypeValue(reader.GetValue(i), reader.GetFieldType(i)));
                        }
                        data.Add(fields);
                    }
                }
                return data;
            }
        }

        // --- MÉTODOS PÚBLICOS OPTIMIZADOS ---

        public string Select(string table, string where = "", string filter = "*")
        {
            string sql = string.Format("SELECT {0} FROM {1}", filter, table);
            if (!string.IsNullOrWhiteSpace(where)) sql += " WHERE " + where;

            ArrayList data = ExecuteBase(sql, ReadToArrayList);
            return JsonConvert.SerializeObject(data);
        }

        public List<T> Select<T>(string table, string where = "", string filter =
"*")
        {
            string json = Select(table, where, filter);
            return JsonConvert.DeserializeObject<List<T>>(json);
        }

        public string SelectSql(string sql)
        {
            ArrayList data = ExecuteBase(sql, ReadToArrayList);
            return JsonConvert.SerializeObject(data);
        }

        public List<T> SelectSql<T>(string sql)
        {
            string json = SelectSql(sql);
            return JsonConvert.DeserializeObject<List<T>>(json);
        }

        public string Insert(string table, List<Hashtable> data)
        {
            string sql = "";
            foreach (var hashdata in data)
            {
                var list = SetStandarValues(hashdata);
                var activeFields = list.Keys.Cast<string>()
                    .Select(k => new { Key = k, Val = GetType(list[k]) })
                    .Where(x => !string.IsNullOrWhiteSpace(x.Val)).ToList();
```

```
                if (activeFields.Any())
                {
                    string fields = string.Join(",", activeFields.Select(x => table
+ "." + x.Key));
                    string values = string.Join(",", activeFields.Select(x =>
x.Val));
                    sql += $"INSERT INTO {table} ({fields}) VALUES ({values});";
                }
            }

            if (!string.IsNullOrEmpty(sql)) ExecuteBase(sql, cmd =>
cmd.ExecuteNonQuery());
            return "Proceso Terminado";
        }

        public T Insert<T>(string table, object data)
        {
            var element =
JsonConvert.DeserializeObject<Hashtable>(JsonConvert.SerializeObject(data));
            string uuid = GetType(element["uuid"] ?? "");
            Insert(table, new List<Hashtable> { element });
            return Select<T>(table, $"uuid = {uuid}").First();
        }

        // Métodos de sobrecarga simplificados
        public string Insert(string table, List<object> data) => Insert(table,
JsonConvert.DeserializeObject<List<Hashtable>>(JsonConvert.SerializeObject(data)));
        public string Insert(string table, object data) => Insert(table,
JsonConvert.DeserializeObject<Hashtable>(JsonConvert.SerializeObject(data)));
        public string Insert(string table, Hashtable data) => Insert(table, new
List<Hashtable> { data });
        public string InsertSql(string sql) => ExecuteBase(sql, cmd => {
cmd.ExecuteNonQuery(); return "Proceso Terminado"; });

        public bool Update(string table, Hashtable data, string where = "")
        {
            data = SetStandarValues(data, 2);
            var updates = data.Keys.Cast<string>()
                .Select(k => new { k, v = GetType(data[k]) })
                .Where(x => !string.IsNullOrWhiteSpace(x.v))
                .Select(x => $"{x.k} = {x.v}");

            if (!updates.Any()) return false;

            string sql = $"UPDATE {table} SET {string.Join(",", updates)}";
            if (!string.IsNullOrWhiteSpace(where)) sql += " WHERE " + where;

            return ExecuteBase(sql, cmd => cmd.ExecuteNonQuery() >= 0);
        }

        public bool Update(string table, object data, string where = "") =>
Update(table,
JsonConvert.DeserializeObject<Hashtable>(JsonConvert.SerializeObject(data)), where);
        public bool UpdateSql(string sql) => ExecuteBase(sql, cmd =>
cmd.ExecuteNonQuery() >= 0);

        public bool SoftDelete(string table, string where = "")
        {
```

```
            string sql = $"UPDATE {table} SET deleted_at = '{DateTime.Now:yyyy-MM-dd
HH:mm:ss}'";
            if (!string.IsNullOrWhiteSpace(where)) sql += " WHERE " + where;
            return ExecuteBase(sql, cmd => cmd.ExecuteNonQuery() >= 0);
        }

        public bool Delete(string table, string where = "")
        {
            string sql = $"DELETE FROM {table}";
            if (!string.IsNullOrWhiteSpace(where)) sql += " WHERE " + where;
            return ExecuteBase(sql, cmd => cmd.ExecuteNonQuery() >= 0);
        }

        public bool DeleteSql(string sql) => ExecuteBase(sql, cmd =>
cmd.ExecuteNonQuery() >= 0);

        public string Call(string method, string parameters) => SelectSql($"EXEC
{method} {parameters}");

        public string CallSql(string sql)
        {
            if (!sql.ToUpper().StartsWith("EXEC ")) sql = "EXEC " +
sql.Replace("CALL ", "");
            return SelectSql(sql);
        }

        // --- MÉTODOS DE CONVERSIÓN ---

        private Hashtable SetStandarValues(Hashtable data, int op = 1)
        {
            data.Remove("id");
            return data;
        }

        private object GetTypeValue(object data, Type type)
        {
            if (data == null || data == DBNull.Value) return null;
            string value = data.ToString().Trim();

            switch (type.Name)
            {
                case "Boolean": return value != "" && Convert.ToBoolean(value);
                case "SByte": return value != "" && Convert.ToInt32(value) == 1;
                case "Int32": case "Int64": case "UInt64": return value != "" ?
Convert.ToInt64(value) : 0;
                case "Decimal": case "Double": return value != "" ?
Convert.ToDouble(value) : 0;
                case "DateTime": return value != "" ? Convert.ToDateTime(value) :
new DateTime();
                default: return value;
            }
        }

        private string GetType(object value)
        {
            if (value == null || value == DBNull.Value) return "NULL";
            switch (value.GetType().Name)
            {
```

```
                case "Boolean": return (bool)value ? "1" : "0";
                case "Double": case "Decimal": return
Convert.ToDouble(value).ToString(nfi);
                case "Int32": case "Int64": return value.ToString();
                case "String": return $"'{value.ToString().Replace("'", "''")}'";
                case "DateTime": return $"'{(DateTime)value:yyyy-MM-dd HH:mm:ss}'";
                default: return $"'{value}'";
            }
        }
    }
}
```